

WashU-2 CPU

Week 2 Report

Sequential Logic — Counters & Clocked Design

Author	Ayush Pati
Roll No.	25B3901
Email	25b3901@iitb.ac.in
Institution	IIT Bombay — Electrical Engineering, Second Year
Programme	SOC 2026 — WashU-2 CPU Mentorship
Submitted	Week 2 · June 2026

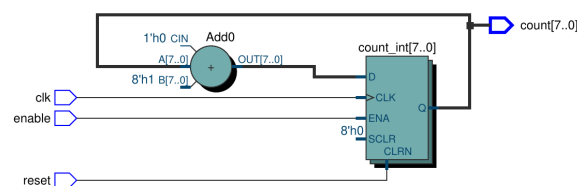
Exercise 1 — 8-bit Up Counter

Entity: counter8

An 8-bit synchronous up counter with active-high asynchronous reset and clock enable. An internal `std_logic_vector` signal **count_int** holds the state; the output port **count** is driven from it via a concurrent assignment. On every rising clock edge, if **enable** = '1' the count increments using a `unsigned()` cast — since VHDL forbids arithmetic directly on `std_logic_vector`. When **reset** = '1' (checked outside `rising_edge`), the counter clears asynchronously.

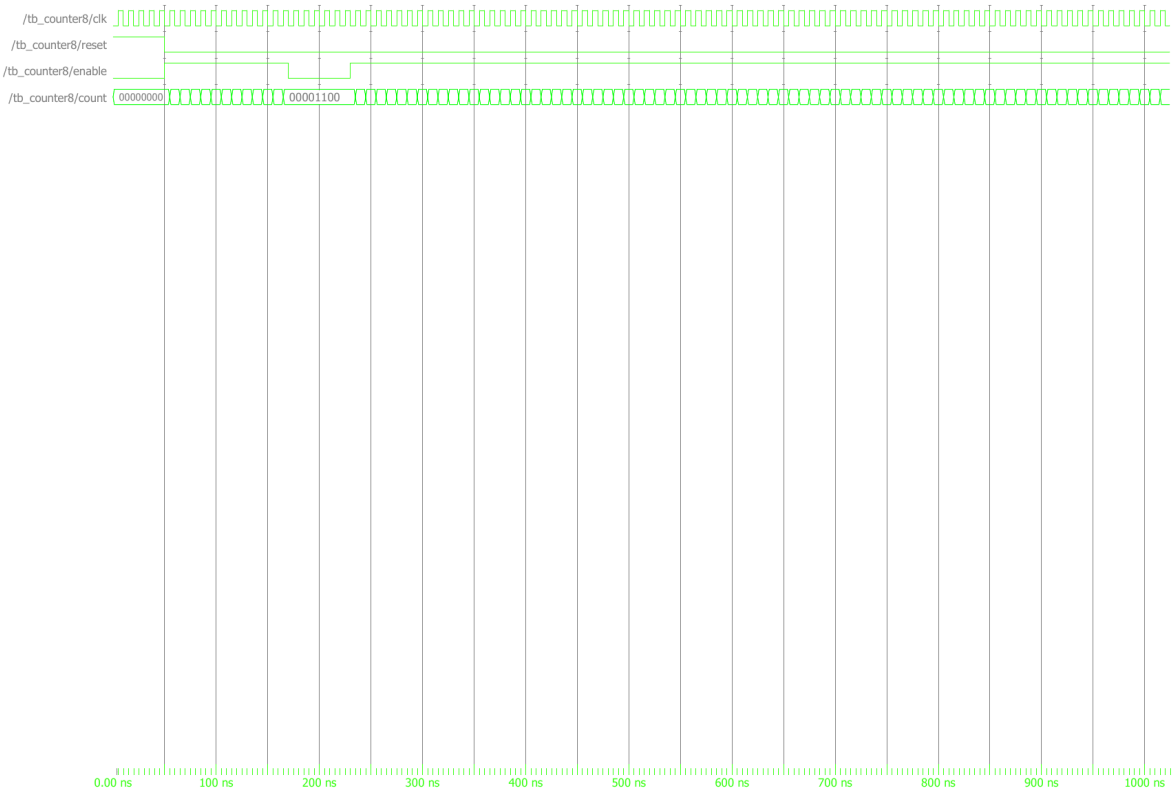
RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.



Entity:tb_counter8 Architecture:sim Date: Thu Jun 18 01:12:03 IST 2026 Row: 1 Page: 1

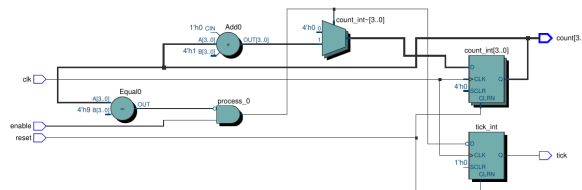
Exercise 2 — Modulo-10 Counter with Tick Output

Entity: mod_counter

A 4-bit counter that counts 0–9 and wraps back to 0, producing a single-cycle **tick** pulse on every rollover. The wrap condition `count_int /= x"9"` keeps the count in range; when the count reaches 9 and enable is high, the counter resets to 0 and **tick_int** is asserted for one cycle. Both count and tick are driven from internal signals to avoid reading output ports — a VHDL-93 restriction. The **tick** output makes this counter chainable as a digit stage in a BCD display or multi-digit counter.

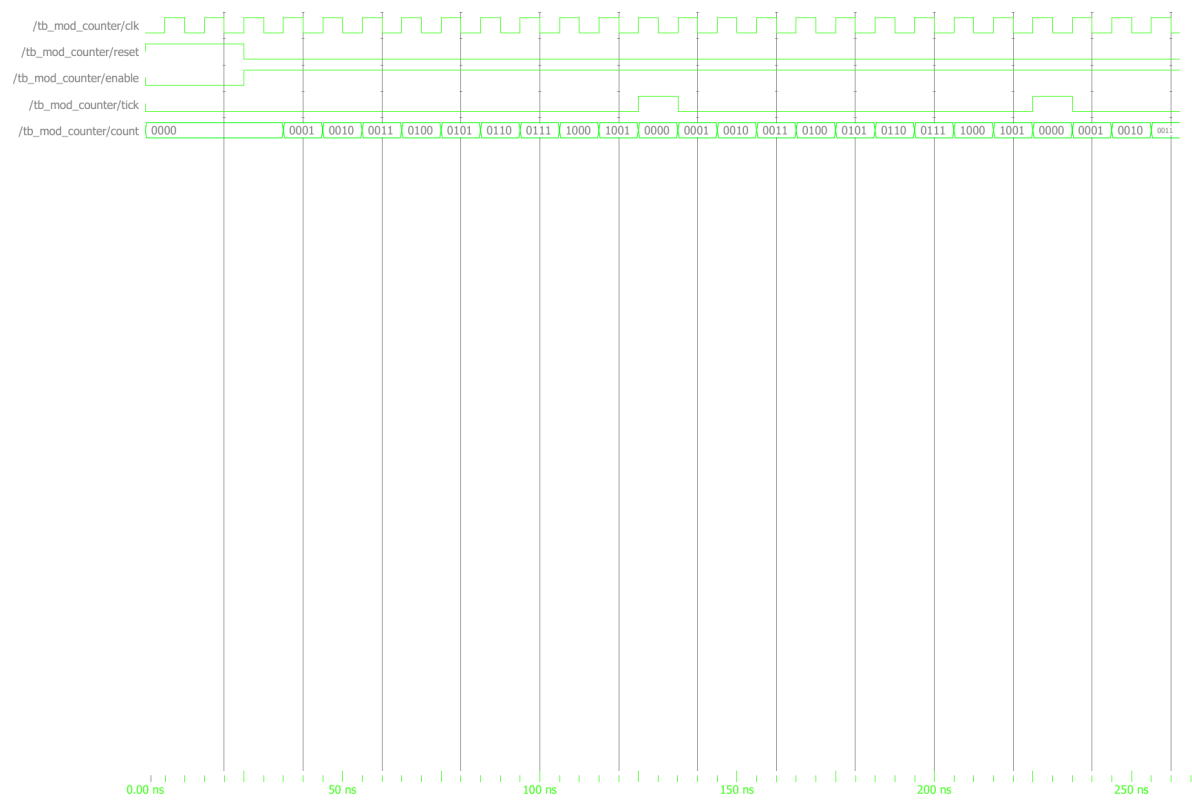
RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.



Entity:tb_mod_counter Architecture:sim Date: Thu Jun 18 03:09:32 IST 2026 Row: 1 Page: 1

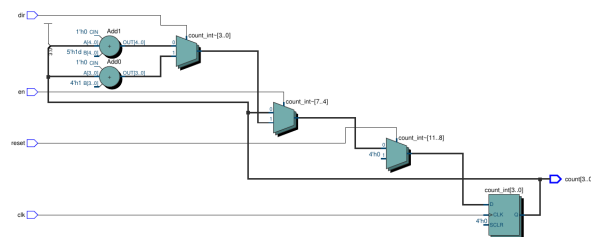
Optional Exercise — 4-bit Bidirectional Counter

Entity: bi_counter4

A 4-bit counter that counts up or down based on a direction control signal **dir**. When **dir** = '1' the counter increments; when **dir** = '0' it decrements. The internal signal is declared as unsigned(3 downto 0) directly — eliminating the cast needed in Exercise 1 — and converted to std_logic_vector only on the output assignment. Reset is checked inside rising_edge(clk), making this a fully synchronous design. Wrapping is natural: decrementing below 0 rolls to x"F" automatically via unsigned arithmetic.

RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.



Bugs & Debugging Notes

All bugs below were encountered while writing and debugging the `tb_mod_counter` testbench. They are rooted in three themes: VHDL operator syntax unfamiliarity, the rules around signal drivers, and correct testbench structure for clocked designs.

Bug 1 — Misunderstanding the `/=` Operator

Encountered `count_int /= x"9"` and was unsure what it meant.

```
count_int /= x"FF" -- unclear: is this assignment? subtraction?
```

Fixed:

```
count_int /= x"FF" -- means: count_int is NOT equal to 0xFF (255)
-- /= is VHDL's inequality operator, equivalent to != in C/C++
-- x"FF" is a hex literal: x"..." prefix = hexadecimal value
```

`/=` is VHDL's not-equal-to operator — the equivalent of `!=` in C. It is used in conditions like `if count_int /= x"9"` then to keep counting while below 9. The `x"..."` prefix denotes a hexadecimal literal, so `x"9"` = 9 decimal = 1001 binary. No code change was needed — this was a reading comprehension gap resolved by consulting the operator table.

Bug 2 — Comma-Separated Signal Assignment

Tried to initialise multiple signals in one statement using commas:

```
clk, count, tick <= '0'; -- ■ syntax error: VHDL has no multi-signal assignment
```

Fixed:

```
clk <= '0';
count <= '0';
tick <= '0';
```

VHDL requires a separate `<=` statement for every signal. There is no comma-separated shorthand like C's `a = b = 0`. The only bulk-assignment idiom is `(others => '0')`, which applies to all bits within a single `std_logic_vector` — it cannot span multiple independent signals.

Bug 3 — Multiple Drivers on clk

The clock signal was driven from two places simultaneously — a concurrent oscillator in the architecture body and an assignment inside the stimulus process:

```
-- Concurrent driver in architecture body:
clk <= not clk after clk_period / 2;

-- Also inside stimulus process: ■
clk <= '0'; -- second driver – conflict!
```

Fixed:

```
-- Signal declaration with initial value (replaces process assignment):
signal clk : std_logic := '0';

-- Single concurrent oscillator (only driver):
clk <= not clk after clk_period / 2;

-- stimulus process: do NOT assign clk here at all
```

VHDL prohibits two processes (or a process and a concurrent statement) from driving the same `std_logic` signal — it creates a multiple-driver conflict that results in 'X' (unknown) in simulation. The fix is to give `clk` an initial value of '0' in its signal declaration and let the single concurrent oscillator statement be the only driver. Remove any `clk <= ...` assignment from inside the process entirely.

Bug 4 — Driving UUT Output Signals from the Testbench

The stimulus process attempted to assign values to **count** and **tick**, which are output ports of `mod_counter`:

```
-- Inside stimulus process:
count <= (others => '0'); -- ■ count is driven by mod_counter
tick <= '0'; -- ■ tick is driven by mod_counter
```

Fixed:

```
-- Only drive INPUT ports of the UUT:
reset <= '1';
enable <= '0';
```

Output ports of the UUT are already driven by the design under test through the port map. A testbench assigning to the same signals creates a second driver — a conflict that collapses to 'X'. The testbench must only drive *input* ports (`clk`, `reset`, `enable`); output signals (`count`, `tick`) are read-only from the testbench's perspective.

Bug 5 — (others => '0') on a Scalar std_logic

The aggregate initialisation syntax was applied to a single-bit std_logic signal:

```
signal count : std_logic;
count <= (others => '0'); -- ■ aggregate invalid on scalar
```

Fixed:

```
count <= '0'; -- ■ for std_logic scalar
-- If a vector is needed:
signal count : std_logic_vector(3 downto 0);
count <= (others => '0'); -- ■ valid for vectors
```

(others => '0') is an aggregate expression valid only for arrays and vectors (std_logic_vector, unsigned, etc.). A bare std_logic is a scalar — assign it directly with '0' or '1'. This bug became moot once Bug 4 was fixed, since count should not be assigned from the testbench at all.

Bug 6 — Stimulus Process Loops Infinitely (Missing wait;)

The stimulus process had no terminal wait; statement, causing it to restart from the top after reaching end process:

```
stimulus : process
begin
reset <= '1';
enable <= '0';
wait for 25 ns;
reset <= '0';
enable <= '1';
wait for 250 ns;
-- ■ no terminal wait – process restarts infinitely!
end process;
```

Fixed:

```
wait for 250 ns;
wait; -- ■ halts the process permanently
end process;
```

A VHDL process without a sensitivity list loops back to begin every time it reaches end process. Without a terminal wait; (no condition — waits forever), the stimulus repeats infinitely: reset fires again, the counter reinitialises, and the waveform never settles into steady-state counting. A bare wait; at the end of the stimulus suspends the process permanently after the test sequence completes.

Bug 7 — std_logic_vector Declared Without Bit-Width

A testbench signal was declared as an unconstrained std_logic_vector with no size range:

```
signal count : std_logic_vector; -- ■ unconstrained – won't compile
```

Fixed:

```
signal count : std_logic_vector(3 downto 0); -- ■ 4-bit, matches mod_counter port
```

```
signal count : std_logic_vector(7 downto 0); -- ■ 8-bit, matches counter8 port
```

std_logic_vector is an unconstrained array type — it has no inherent width. Every signal declaration must include an explicit range such as (7 downto 0). The range must match the width of the UUT port it connects to in the port map; a mismatch causes a type-size error at elaboration. Checking the entity declaration of the design under test for the port width is the first step before writing any testbench signal declarations.